ENGINEERING WORKSHOP · ISAAC LAB 3 + NEWTON

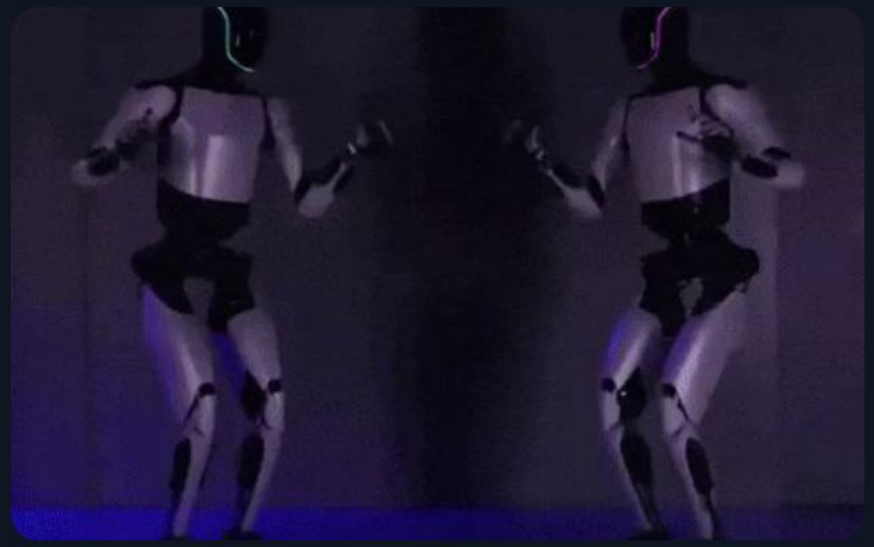
Simulation for Humanoid

Week 1 — Introduction to Humanoid

Isaac Lab 3.0 + Newton 2026 edition

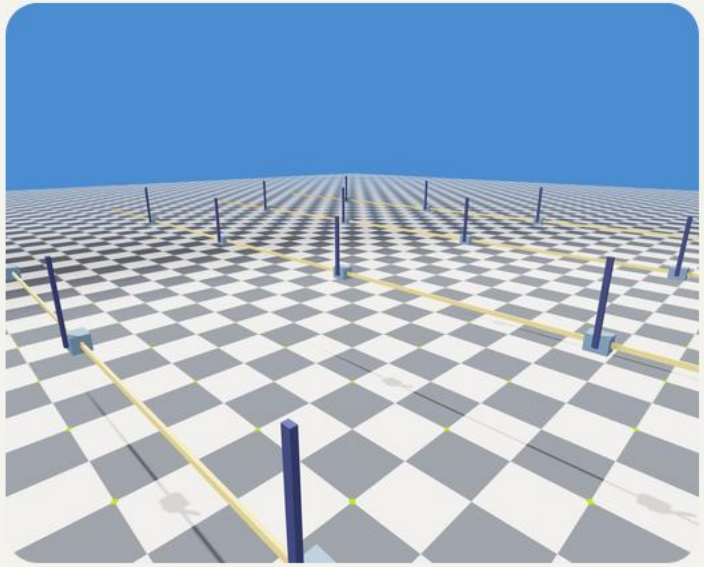
Pannatorn Bongkotamphon
Dr. Sukhum Sattaratnamai





WEEK 1

Agenda



01 Preparation

ติดตั้ง Isaac Lab 3 + Newton ด้วย uv — ไม่ต้องมี conda

02 First training

Cartpole: train → play ใน Newton viewer

03 Intro to Humanoid

ทำไมต้องหุ่นมนุษย์ · ทำไมต้อง Simulation

04 Exercises

scaling · Hydra override · ablation · RL library



THIS WEEK

Simulation machine preparation

ติดตั้งก่อน — แล้วคุยเรื่องหุ่นยนต์ตอนรอโหลด

uv

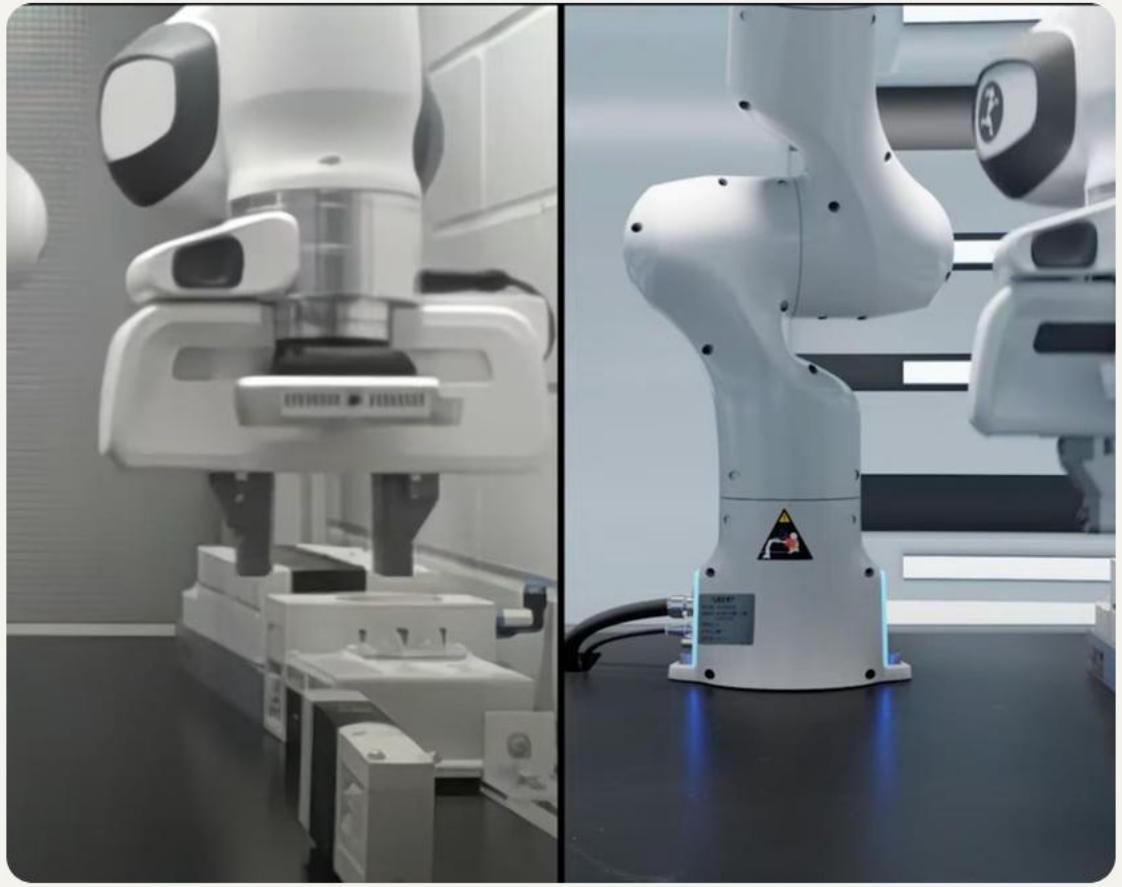
Isaac Lab 3.0

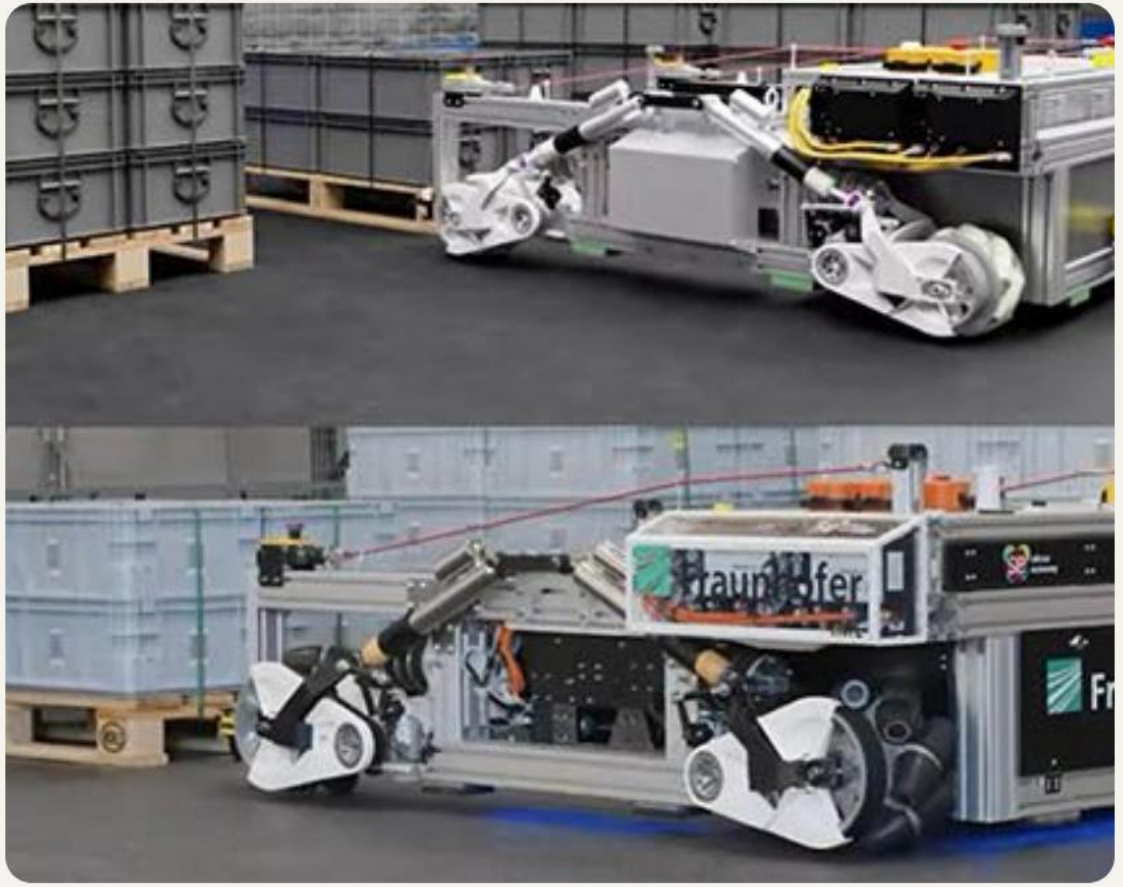
Newton physics

~~conda · pip isaacsim~~

QUIZ

อันไหนของจริง อันไหน Simulation?





WHAT CHANGED IN 2026

Isaac Lab 2.x → 3.0 + Newton

เรื่อง	เดิม (ม.ค. 2026)	ใหม่ (2026 edition)
ติดตั้ง	conda + pip isaacsim (~1 ชม.)	uv run — สร้าง env ให้อัตโนมัติ
Physics	PhysX ใน Isaac Sim (Kit)	Newton (MJWarp) · PhysX เลือกได้
GPU	แนะนำ 16 GB VRAM	เทรนได้บน RTX 3060 Laptop 6 GB
Python	3.11	3.12 (uv จัดการให้)
คำสั่ง	./isaaclab.sh -p .../rsl_rl/train.py	uv run isaaclab train --rl_library rsl_rl
Task	Isaac-Velocity-Flat-Spot-v0	IsaacContrib-Velocity-Flat-Spot
หน้าต่าง	GUI เสมอ · ใส่ --headless	headless เป็นค่าเริ่มต้น · ใส่ --viz newton

ที่มา: Isaac Lab docs “Migrating to 3.0” · ทดสอบจริงบน Isaac Lab develop (3.0.0), ต.ค. 2026

OUR PLATFORM

โค้ดชุดเดียว เลือก physics ได้

Isaac Lab 3.0

Tasks · Rewards · Terrain · RL (rsl_rl, skrl, sb3, rl_games)

↓ physics=… ↓

ค่าเริ่มต้น · workshop นี้ใช้

Newton

open-source GPU physics บน Warp · ไม่ต้องมี Omniverse Kit

`uv run isaaclab …`

ทางเลือก

OV PhysX

PhysX แบบ standalone

`--extra ovphysx`

หนัก · 16 GB VRAM

Isaac Sim

Kit app + RTX rendering

`--extra isaacsim`

Viewer: newton · rerun · viser · kit

NEWTON กับ MUJOCO มาจากไหน

จาก MuJoCo บน CPU สู่ Newton บน GPU

2012 — **MuJoCo**
Emo Todorov (Univ. of Washington) · physics สำหรับ robot control · ขายไลเซนส์

2021–22 — **MuJoCo ฟรี + open source**
Google DeepMind ซื้อ (ต.ค. 2021) แจกฟรี · เปิดโค้ด Apache-2.0 (พ.ค. 2022)

2022 — **NVIDIA Warp**
เขียน Python แล้ว compile เป็น CUDA kernel รันบน GPU · มี warp.sim ในตัว

2023 — **MJX**
MuJoCo บน JAX (MuJoCo 3.0) — ก้าวแรกของ MuJoCo บน GPU/TPU

2025 — **Newton**
Disney Research + Google DeepMind + NVIDIA · ต่อยอด warp.sim · ใช้ MuJoCo Warp เป็นแกนหลัก

2026 — **Isaac Lab 3**
ใช้ Newton เป็น physics หลัก (physics=newton_mjwarp) · Newton 1.6 เป็นโครงการของ Linux Foundation



MuJoCo — โลโก้จาก github.com/google-deepmind/mujoco



Newton — H1 สวมเสื้อจำลอง (example_cloth_h1) · ภาพ Newton docs, CC-BY-4.0

ที่มา: README ของแพ็กเกจ newton 1.6.0, mujoco 3.12.0, mujoco-warp 3.12.0, warp-lang 1.17.0 ที่ติดตั้งในเครื่อง workshop · ประวัติ MuJoCo จาก mujoco.org

สัมพันธ์กันยังไง

Newton = กรอบ · MuJoCo Warp = เครื่องยนต์ข้างใน

Isaac Lab 3 — task · reward · RL · Hydra override

Newton — โหลด USD · สร้างหุ่นพันตัว · contact · viewer · เลือก solver

- **MuJoCo Warp** — หุ่นยนต์ข้อต่อ + contact · ค่าเริ่มต้น
- **Featherstone** — articulation แบบคลาสสิก
- **XPBD** — position-based · เร็ว
- **VBD** — ผ้า / soft body
- **Implicit MPM** — ทราย ดิน วัสดุเป็นเม็ด
- **Kamino** — solver ใหม่ (preset newton_kamino)

NVIDIA Warp — Python → CUDA kernel · Newton และ MuJoCo Warp เขียนด้วย Warp

MuJoCo (เดิม) = C/C++ บน CPU · **MuJoCo Warp** = algorithm เดียวกัน เขียนใหม่ด้วย Warp ให้รันหลายพันฉากพร้อมกันบน GPU

physics=newton_mjwarp = Newton + solver MuJoCo Warp · **physics=physx** = สลับไปใช้ PhysX ได้ด้วยโค้ด task เดิม

NEWTON ทำอะไรได้บ้าง · EXAMPLE ที่มากับแพ็กเกจ

solver ต่างกัน = จำลองของคนละแบบ



MuJoCo Warp

ANYmal C เดินด้วย policy ที่เทรนแล้ว



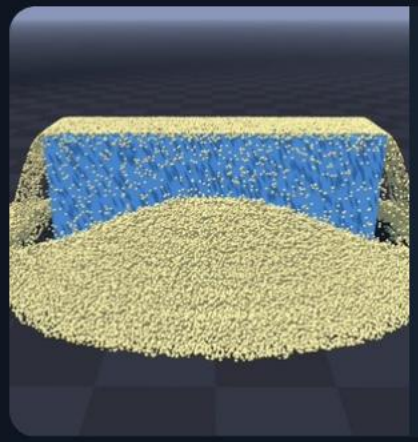
MuJoCo Warp

G1 humanoid หลายตัวพร้อมกัน



Kamino

ANYmal D ด้วย solver ตัวใหม่



Implicit MPM

ทรายไหลออกจากกล่อง



VBD

soft body ห้อยแล้วโค้งตามน้ำหนัก



Featherstone + VBD

แขน Franka (rigid) ยกผ้า (soft) — ผสม solver

ภาพ: Newton documentation (github.com/newton-physics/newton, CC-BY-4.0) · ชื่อ solver ตรวจจากโค้ด example ใน newton 1.6.0 · ลองเองได้: `uv run python -m newton.examples robot_g1`

BEFORE ISAAC LAB · ISAAC SIM 101

Isaac Sim กับ Isaac Lab ต่างกันยังไง

ฐาน

Omniverse Kit

แพลตฟอร์ม 3D ของ NVIDIA ที่ทำงานบน USD + RTX rendering



SIMULATOR · มี GUI

Isaac Sim

เปิดฉาก จัดวางหุ่น ใส่ฟิสิกส์ (PhysX) กด Play ดูผล — ทีละฉาก ด้วยมือ



FRAMEWORK · โค้ด

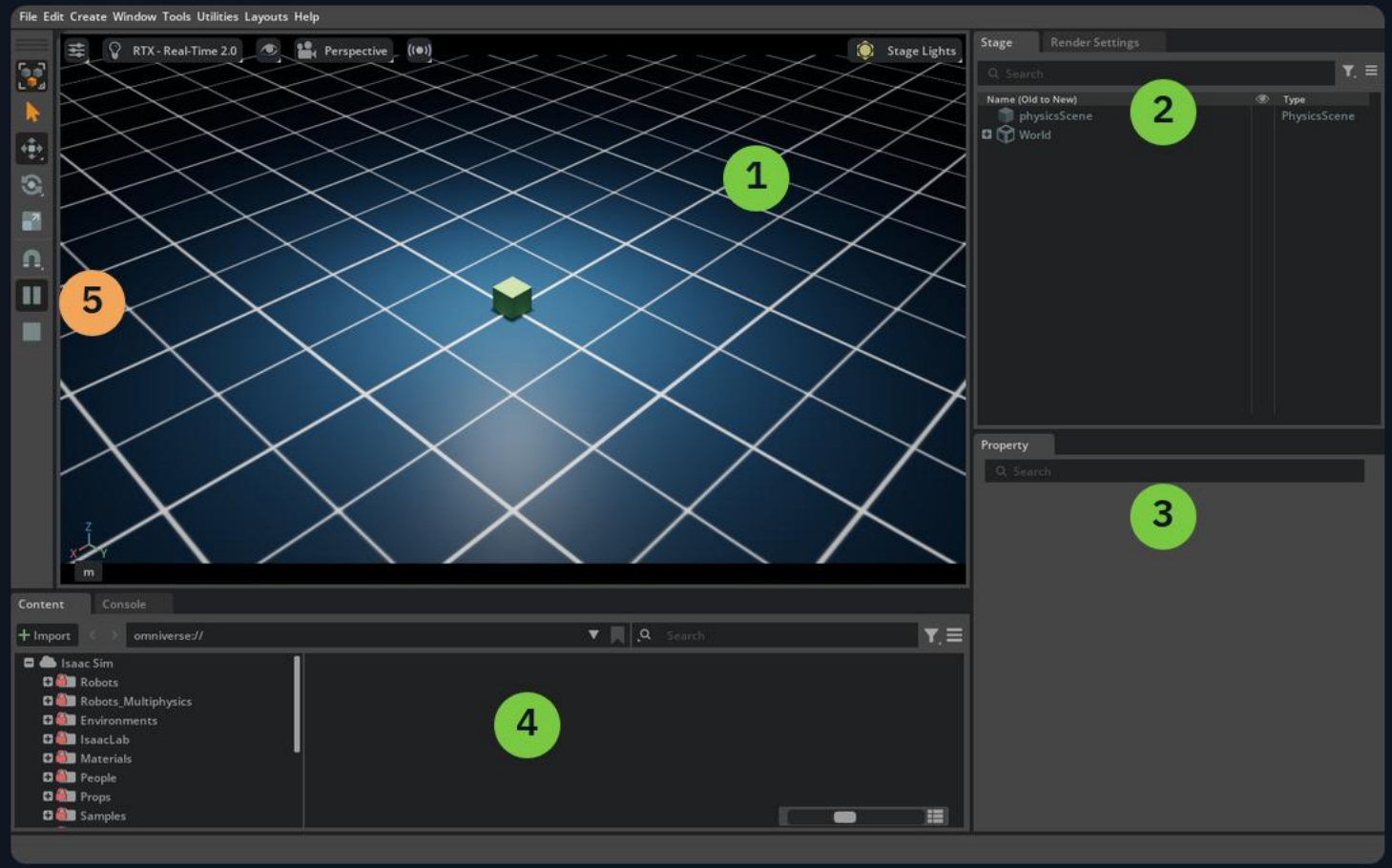
Isaac Lab

เขียน task + reward เป็น Python แล้วเทรน RL หุ่นพันตัวพร้อมกัน

Isaac Lab 3: ไม่ต้องมี Isaac Sim แล้ว — เลือก physics เป็น **Newton** ได้โดยตรง แต่ Isaac Sim ยังมีประโยชน์ตอน “ดูและแก้ฉาก” ด้วยตา

ISAAC SIM 101 · หน้าจอหลัก

ทัวร์หน้าจอ Isaac Sim 6.1



1 Viewport — ภาพ 3D (RTX) หมุน/ซูมด้วยเมาส์

2 Stage — ต้นไม้ของ prim ใน USD (World, physicsScene, หุ่น)

3 Property — ค่าของ prim ที่เลือก: transform, mass, joint

4 Content — asset สำเร็จรูป: Robots, Environments, IsaacLab

5 Play / Pause — เริ่มฟิสิกส์ (ในภาพ: กล่องตกลงมาวางบนพื้นแล้ว)

ภาพจริงจากเครื่อง workshop (RTX 3060 Laptop 6 GB) · ฉาก: ground plane + DynamicCuboid

ISAAC SIM 101 · วัดจริงบนเครื่อง WORKSHOP

ทำไม workshop นี้เทรนด้วย Newton

RTX 3060 Laptop 6 GB	Isaac Sim 6.1 (GUI ผ่าน SimulationApp)	Isaac Lab 3 + Newton
เปิดครั้งแรก	~100 s + compile shader ~13 นาที	~43 s (cache ยังว่าง · วัดใน Week 1)
เปิดครั้งต่อไป	~85 s	~15 s ก่อนเริ่มเทรน
ความเร็ว	~18–19 fps · ฉากกล่อง 1 ใบ	~58,000 steps/s · หุ่น 2,048 ตัว (เทรน)
ใช้ทำอะไร	ดู/แก้ฉาก USD ด้วยตา, กล้อง RTX, Physics Inspector	เทรน RL, ทดลองหลายค่าเร็ว ๆ
Import URDF ใน GUI	ใช้ไม่ได้บนเครื่องนี้ (ขาด libxml2.so.2)	urdf_to_usd.py (Week 9) ใช้ได้

แนวทาง: ใช้ Isaac Sim “ดู” ฉากหรือหุ่นเป็นครั้งคราว · เทรนด้วย Isaac Lab + Newton

แอปเต็ม `./.venv/bin/isaacsim` โหลดเกิน 3 นาทีบนเครื่องนี้ — ใช้สาธิตบนเครื่องผู้สอนที่แรงกว่า

ISAAC SIM 101 · เขียนโปรแกรม

สั่ง Isaac Sim ด้วย Python ได้ 2 ทาง

Window › Script Editor

1 · พิมพ์ในหน้าต่างที่เปิดอยู่

รันโค้ดกับฉากตรงหน้าทันที เหมาะลอง API ทีละบรรทัด · ไม่ต้องสร้าง SimulationApp

./.venv/bin/python hello.py

2 · standalone script

เปิด Isaac Sim เองจากไฟล์ .py · มีหรือไม่มีหน้าต่างก็ได้ (headless) · ใช้ทำงานซ้ำ ๆ / อัตโนมัติ

```
from isaacsim import SimulationApp
app = SimulationApp({"headless": True}) # ต้องมาก่อน import อื่น
import numpy as np
from isaacsim.core.api import World
from isaacsim.core.api.objects import DynamicCuboid
world = World()
world.scene.add_default_ground_plane()
cube = world.scene.add(DynamicCuboid(prim_path="/World/Cube",
    name="cube", position=np.array([0, 0, 1.0]), size=0.3))
world.reset()
for i in range(121):
    world.step(render=True) # 1 step = 1/60 s
    if i % 30 == 0: print(cube.get_world_pose()[0][2])
app.close()
```

ผลจริง · ISAAC SIM 6.1

```
t=0.05 s  z=0.992 m
t=0.55 s  z=0.285 m
t=1.05 s  z=0.150 m
t=2.05 s  z=0.150 m
```

กล่องตกถึงพื้นใน ~1 s แล้วหยุดที่ 0.15 m = ครึ่งหนึ่งของขนาด 0.3 m

Isaac Lab ทำงานแบบเดียวกันข้างใน แต่สร้างหุ่นหลายพันตัวพร้อมกันและจัดการ step ให้

ISAAC SIM 101 · ACTION GRAPH (OMNIGRAPH)

เขียนโปรแกรมแบบต่อกล่อง ไม่ต้องพิมพ์โค้ด



เปิดจาก **Window › Graph Editors › Action Graph** · ลากกล่อง (node) มาต่อสาย: เส้น “exec” บอกว่าเมื่อไหร่ทำ · เส้นข้อมูลส่งค่าไปตามกล่อง

```
keys = og.Controller.Keys
og.Controller.edit({"graph_path": "/World/SpinGraph", "evaluator_name": "execution"}, {
  keys.CREATE_NODES: [
    ("Tick", "omni.graph.action.OnPlaybackTick"),  ("Deg", "omni.graph.nodes.Multiply"),
    ("Rot", "omni.graph.nodes.MakeVector3"),
    ("Write", "omni.graph.nodes.WritePrimAttribute")],
  keys.CONNECT: [
    ("Tick.outputs:tick", "Write.inputs:execIn"),  # exec: ทำทุกเฟรม
    ("Tick.outputs:time", "Deg.inputs:a"),  ("Deg.outputs:product", "Rot.inputs:z"),
    ("Rot.outputs:tuple", "Write.inputs:value")],
  keys.SET_VALUES: [
    ("Deg.inputs:b", {"type": "double", "value": 90.0}),
    ("Write.inputs:prim", [usdrt.Sdf.Path("/World/Spinner")]),
    ("Write.inputs:name", "xformOp:rotateXYZ")]})
```

ผลจริง

```
t=0.55 s → 49.5°
t=1.05 s → 94.5°
t=2.05 s → 184.5°
```

หมุน 90° ต่อวินาทีตามที่ต่อไว้

ใช้จริงบ่อย: ROS 2 bridge (ส่ง clock, joint state), Articulation Controller สั่งข้อต่อหุ่น, รับปุ่มคีย์บอร์ด

RL LIBRARIES ใน ISAAC LAB 3

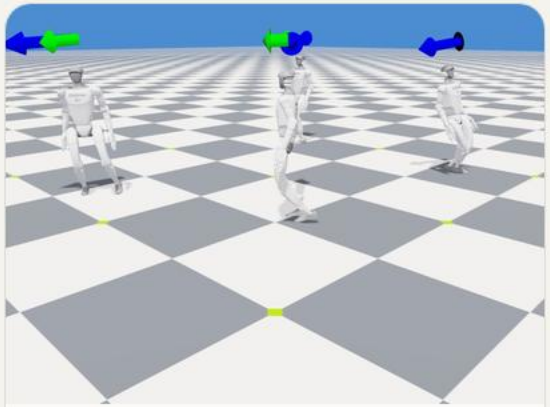
env เดียว · เลือก library ได้ 5 ตัว

library	ผู้พัฒนา	จุดเด่น	task ที่มี config	เหมาะกับ
`rsl_rl`	ETH Zurich · Robotic Systems Lab (ANYmal)	PPO ทั้งหมดอยู่บน GPU · distillation (teacher → student) · symmetry · RNN	**90**	หุ่นขา · humanoid locomotion — **workshop นี้ใช้**
`rl_games`	Denys Makoviichuk (NVIDIA) · Isaac Lab ใช้ fork isaac-sim/rl_games	PPO throughput สูง · asymmetric actor-critic · LSTM	**42**	มือหุ่นยนต์ (Shadow, Allegro) · Factory · หยิบจับ
`skrl`	Antonio Serrano-Muñoz (community)	algorithm หลายตัว · PyTorch **และ** JAX · AMP · multi-agent (IPPO, MAPPO)	**49**	ลอง algorithm อื่น · หลาย agent · AMP (ท่าทางจาก motion capture)
`sb3`	Stable-Baselines3 · DLR-RM	API มาตรฐาน · PPO, SAC, TD3 · คู่มือ (documentation) ละเอียด มีตัวอย่างครบ	**7**	เรียน RL · ต้นแบบ — แต่ส่งข้อมูลผ่าน numpy บน CPU ทุก step
`torchrl`	PyTorch	ชิ้นส่วนสำหรับประกอบ RL เอง	**2**	ทีมที่สร้าง pipeline เอง (ตัวอย่างใน Isaac Lab มีแค่ Cartpole)

นับจาก `*_cfg_entry_point` ใน isaaclab_tasks (Isaac Lab 3.0 develop, ต.ค. 2026) · มี env ทั้งหมด 143 ตัว · rsl_rl-lib 5.5.1 · skrl ≥ 2.1 · sb3 ≥ 2.6

ตัวอย่างงานจริง · เปิดบนเครื่อง WORKSHOP

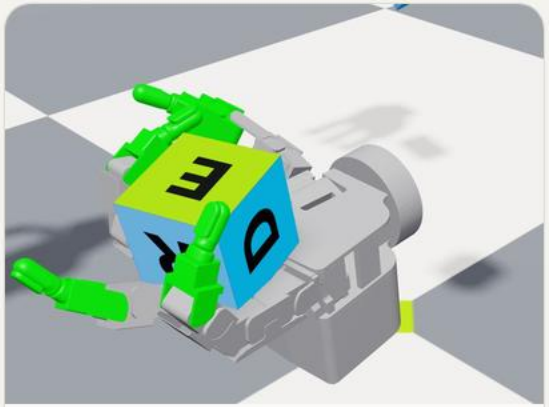
แต่ละ library ถนัดงานแบบไหน



rsl_rl

`Isaac-Velocity-Flat-G1`

เดินด้วยขา: humanoid / หุ่นสี่ขา · obs จาก IMU + ข้อต่อ · สั่งความเร็ว



rl_games

`Isaac-Reorient-Cube-Allegro`

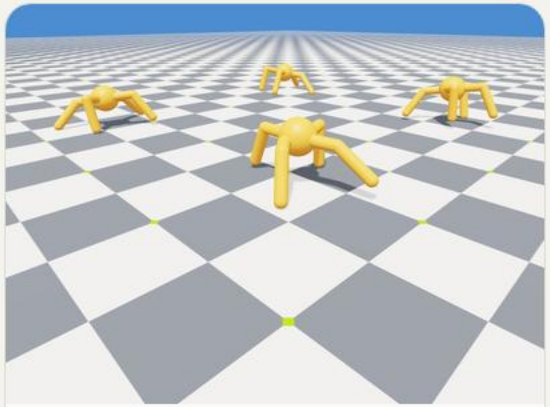
มือ 16 ข้อหมุนลูกเต๋าให้ตรงกับเป้า (ลูกเต๋าลอยมุมบน = ท่าที่ต้องการ)



skrl

`Isaac-Shadow-Handover`

มือ 2 ข้าง = 2 agent ส่งบอลให้กัน · เทรนแบบ multi-agent (IPPO / MAPPO)



sb3

`Isaac-Ant`

หุ่นง่าย obs น้อย · เหมาะลอง algorithm (PPO, SAC) ก่อนขยับไปงานใหญ่

ภาพจาก Newton viewer บนเครื่องนี้ · ยังไม่ได้เทรน (action สุ่ม) · task ทุกตัวในภาพเปิดด้วย library อื่นได้ด้วย — ที่เขียนคือ library ที่ตัวอย่างของ Isaac Lab ถนัด · AMP และ Factory (หยิบประกอบชิ้นส่วน) ยังรันได้เฉพาะ PhysX ใน Isaac Sim

เลือกยังไง

ทำไม workshop นี้ใช้ rsl_rl

มี config ให้ทุก task ที่เราใช้

Cartpole, Go2, Spot, G1, H1 มี rsl_rl_cfg_entry_point ครบ — ไม่ต้องเขียน config PPO เอง

ข้อมูลไม่ออกจาก GPU

obs / reward เป็น torch tensor บน GPU ตลอด · 4,096 env ไม่ต้อง copy ไป CPU

ครบสำหรับ sim-to-real

distillation, symmetry, export เป็น JIT / ONNX (Week 9 ใช้ ONNX)

งานแบบนี้ → ใช้

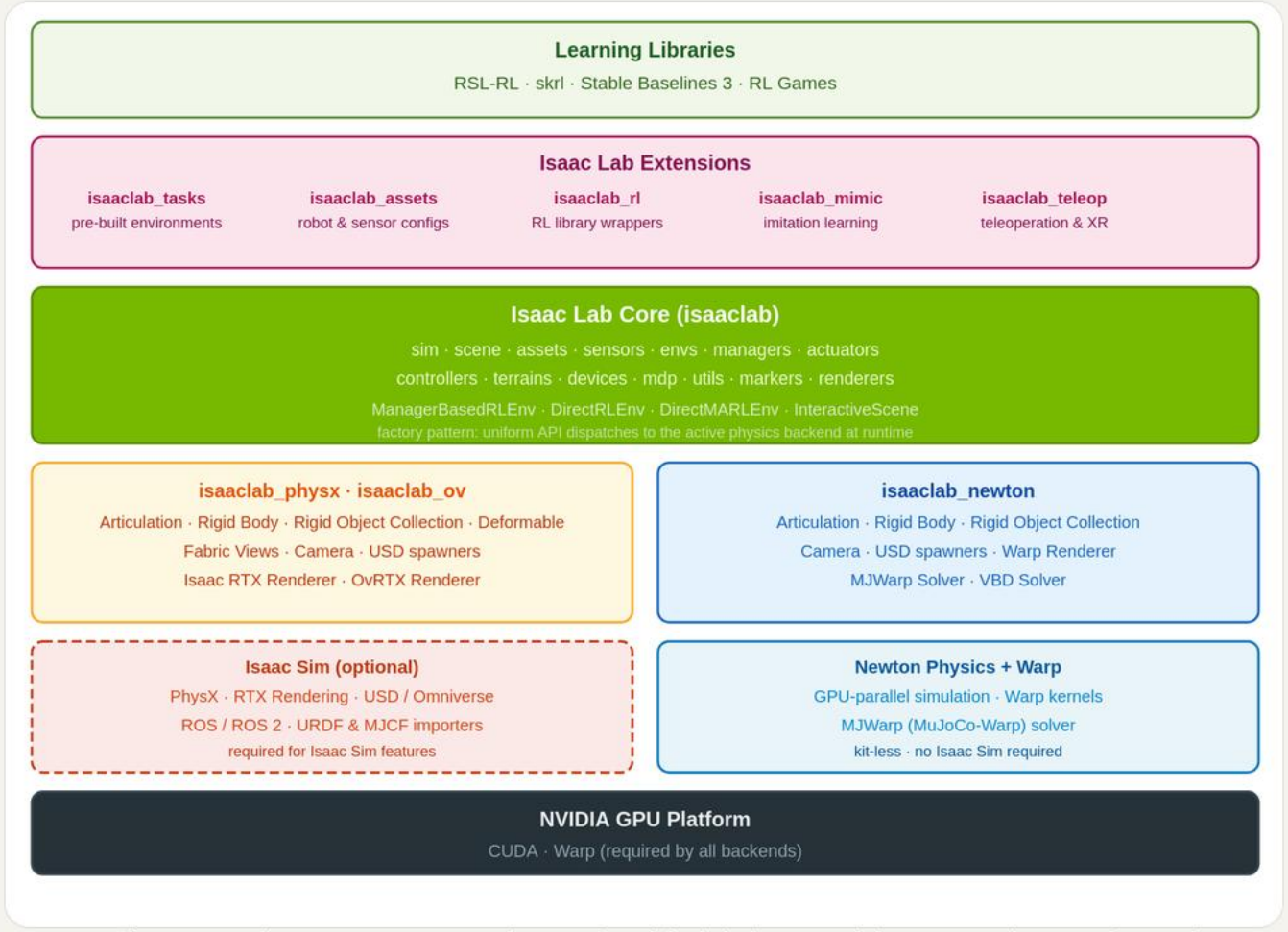
หุ่นขา · humanoid เดิน	`rsl_rl`
มือหยิบ · หมุนวัตถุ	`rl_games`
หลาย agent · AMP · JAX	`skrl`
เรียน algorithm · SAC/TD3 · env น้อย	`sb3`

```
uv run --extra skrl isaaclab train \
  --rl_library skrl --task …
```

ทดลองเปลี่ยน library จริงใน **Exercise 1.4** — task เดิม ผลใกล้กันมาก ต่างกันที่ความเร็วและวิธีแสดงผล

FROM THE DOCS · ISAAC LAB ECOSYSTEM

Isaac Lab วางอยู่ตรงไหน



“Isaac Lab ไม่ใช่ simulator — เป็น framework สำหรับ robot learning ที่วางบน simulator”

1 บนสุด: RL libraries

rsl_rl · skrl · sb3 · rl_games — ตัวที่เราเลือกตอนรันคำสั่ง

2 กลาง: isaaclab core

scene · managers · envs · sensors — โค้ด task ของเรา

3 ล่าง: physics 2 ทาง

PhysX (ต้องมี Isaac Sim) หรือ **Newton + Warp** แบบ kit-less ← workshop นี้

สรุปจากหน้า Isaac Lab Ecosystem · ภาพ: isaac-sim.github.io/IsaacLab/v3.0.0-EA/source/setup/ecosystem.html

REQUIREMENTS

เครื่องแบบไหนใช้ได้?

Newton (workshop นี้)

- NVIDIA GPU — ทดสอบแล้วบน RTX 3060 Laptop 6 GB
- NVIDIA driver ≥ 580
- Ubuntu 22.04 / 24.04
- Python 3.12 — uv ติดตั้งให้เอง
- Disk เผื่อ ~30 GB

Isaac Sim เต็มตัว

- GPU 16 GB VRAM ขึ้นไป
- RAM 32 GB ขึ้นไป
- ใช้เมื่อต้องการ RTX, กล้อง, USD
- ไม่บังคับใน workshop นี้

ที่มา: Isaac Lab installation docs · เครื่องทดสอบ: driver 595.84, CUDA 13.2

WHAT TO INSTALL

เหลือแค่ 2 อย่าง

1

uv

ตัวจัดการ Python + environment ที่เร็วมาก แทน conda + pip

2

Isaac Lab

git clone แล้ว uv run ลง PyTorch, Newton, RL libs ให้เอง

✖

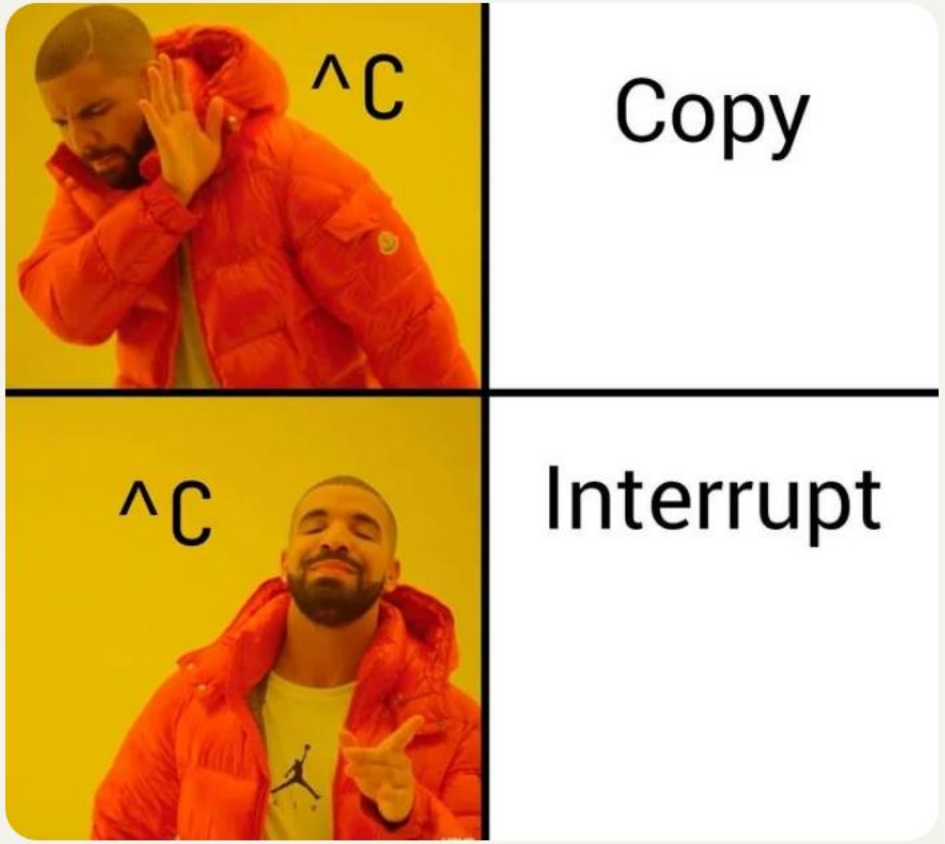
~~conda~~

ไม่ต้องแล้ว — ไม่ต้อง activate env ก่อนทุกครั้ง

LINUX BASICS

Terminal ใช้บ่อยมาก

ทำอะไร	Shortcut
เปิด Terminal	`Ctrl + Alt + T`
หยุดคำสั่ง	**`Ctrl + C`**
Copy	`Ctrl + Shift + C`
Paste	`Ctrl + Shift + V`
ค้นคำสั่งเก่า	`Ctrl + R`



INSTALLATION · STEP 1 / 5

ติดตั้ง uv

01

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
# ปิดแล้วเปิด Terminal ใหม่
$ uv --version
uv 0.12.17 (x86_64-unknown-linux-gnu)
```

Checkpoint: uv --version ขึ้นเลขเวอร์ชัน

ไม่ต้องลง conda / Miniconda แล้ว

INSTALLATION · STEP 2 / 5

Clone Isaac Lab ไว้ที่ home

02

```
$ cd ~
$ git clone https://github.com/isaac-sim/IsaacLab.git \
    --branch develop
$ cd IsaacLab
```

FROM NOW ON

ทุกคำสั่งรันจาก **~/IsaacLab** ด้วย **uv run …** — แทน conda activate แบบเดิม

INSTALLATION · STEP 3 / 5

รันครั้งแรก = ติดตั้งอัตโนมัติ

03

```
$ uv run isaaclab list_envs \
    --keyword Velocity

Isaac-Velocity-Flat-G1
Isaac-Velocity-Rough-G1
Isaac-Velocity-Flat-H1
Isaac-Velocity-Rough-H1
Isaac-Velocity-Flat-UnitreeGo2
IsaacContrib-Velocity-Flat-Spot
…
```

ครั้งแรกนานหน่อย

uv สร้าง .venv แล้วโหลด PyTorch (CUDA 13), Warp, Newton — ทำครั้งเดียว

140 tasks

ไม่ใส่ --keyword จะเห็นทั้งหมด

Checkpoint: เห็นรายชื่อ task = ติดตั้งสำเร็จ

INSTALLATION · STEP 4 / 5 · LAB 1.1

เทรนครั้งแรก: Cartpole

04

```
$ uv run isaaclab train --rl_library rsl_rl --task Isaac-Cartpole
```

```
Task          Isaac-Cartpole
RL library    rsl_rl
Physics       newton_mjwarp
Visualizer    none (headless)
Device        cuda:0
Environments  4096
```

```
Learning iteration 149/150
Steps per second: 314748
Mean episode length: 300.00
success_rate: 1.0000
Training time: 31.59 seconds
```

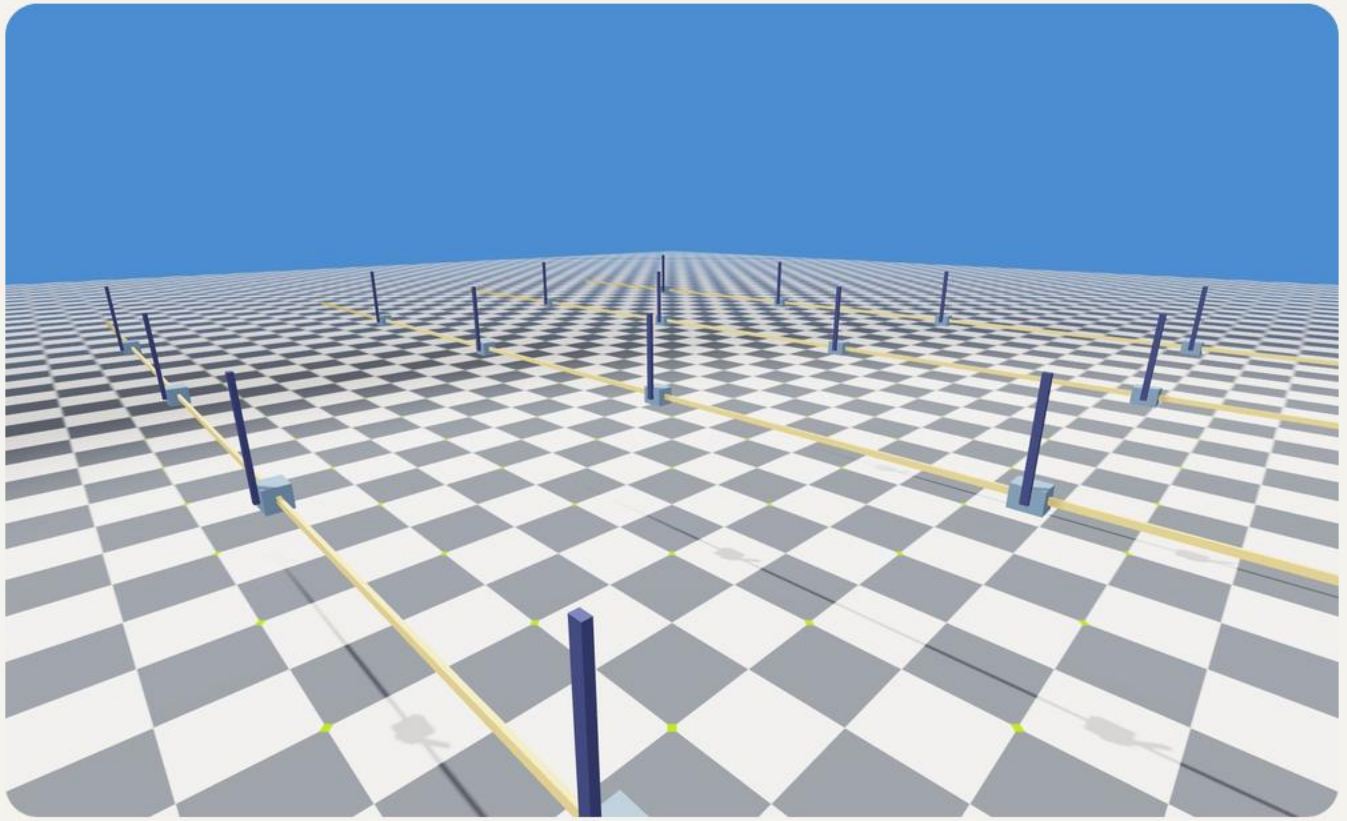
วัดจริงบน RTX 3060 Laptop 6 GB · ไม่มีหน้าต่างเด้ง = ปกติ · เสร็จเมื่อเห็น “Training time”

05

เปิดหน้าต่างดูผล

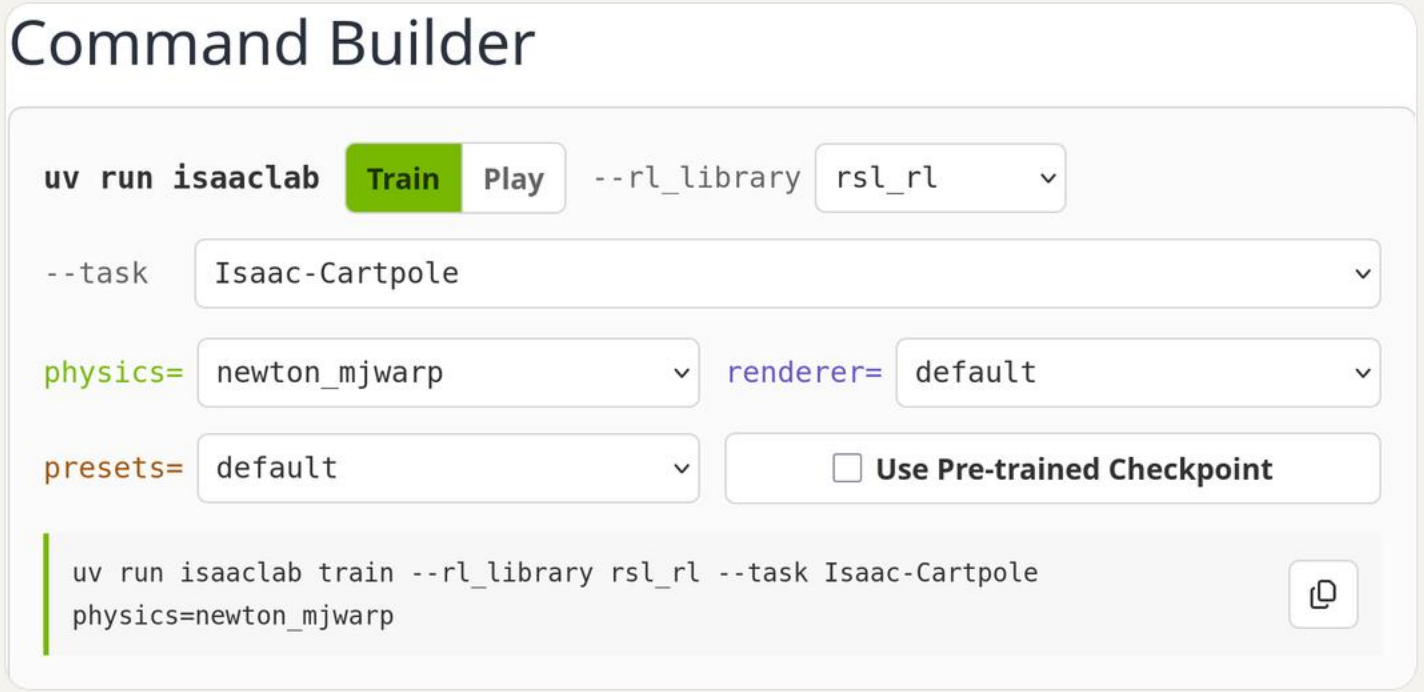
```
$ uv run isaaclab play \
  --rl_library rsl_rl \
  --task Isaac-Cartpole \
  --num_envs 16 \
  --checkpoint latest \
  --viz newton
```

เมาส์ซ้าย หันกล้อง · กลาง orbit · ขวา ลากหุ่น
WASD/QE เลื่อน · F โฟกัส · Space หยุด



FROM THE DOCS · COMMAND BUILDER

ไม่ต้องจำคำสั่ง — ติ๊กแล้วก๊อป



1 **เลือก Train / Play**
และ --rl_library

2 **เลือก --task**
dropdown มี core tasks ทั้งหมด

3 **physics = newton_mjwarp**
renderer / presets ปล่อย default

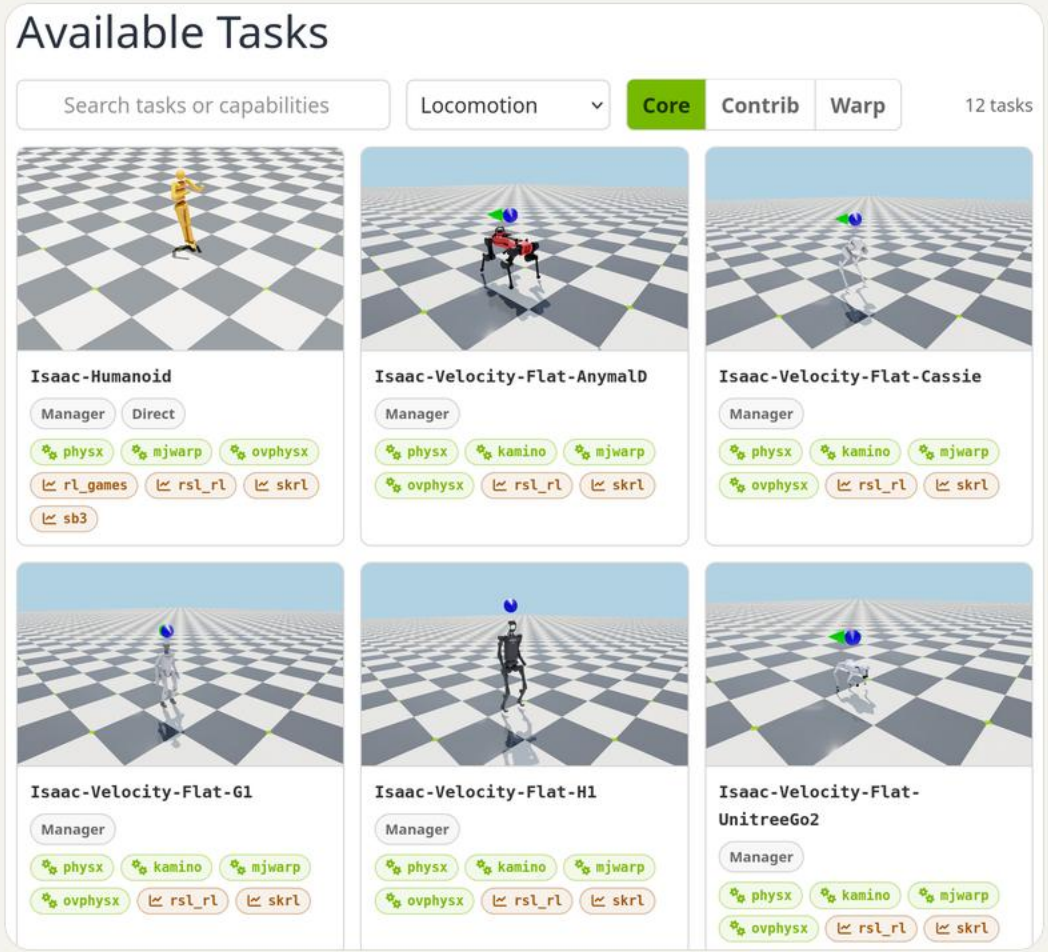
4 **กดปุ่ม copy**
วางใน terminal ที่ ~/IsaacLab

5 **เติมเองถ้าต้องการ**
--viz newton · --num_envs · --max_iterations

ได้คำสั่งเดียวกับ Step 4 · physics=newton_mjwarp ใส่หรือไม่ใส่ก็ได้สำหรับ core tasks · ภาพ: isaac-sim.github.io/IsaacLab/v3.0.0-EA/source/setup/environments.html

FROM THE DOCS · AVAILABLE TASKS

หา task จาก catalog — อ่าน badge ให้เป็น



1 **Core · Contrib · Warp**

แท็บแยกชุด task — Spot อยู่ใน Contrib

2 **ค้นหา + category**

พิมพ์ชื่อหุ่น หรือเลือก Locomotion / Manipulation

3 **Badge เขียว = physics**

physx · kamino · mjwarp · ovphysx — ต้องมี mjwarp ถึงรันบน Newton ของเราได้

4 **Badge ส้ม = RL library**

rsl_rl · skrl · sb3 · rl_games ที่มี config ให้แล้ว

5 **Manager / Direct**

สองแบบการเขียน env: แยกเป็น manager (ประกอบชิ้น) หรือเขียนตรงในคลาสเดียว

Core + Locomotion = 12 tasks (ณ 9 ต.ค. 2026) · ภาพ: isaac-sim.github.io/IsaacLab/v3.0.0-EA/source/setup/environments.html

OPTIONAL

อยากได้ Isaac Sim เต็มตัว?

```
# PhysX + Kit app + RTX rendering (ใช้ VRAM เยอะ)
$ uv run --extra isaacsim isaaclab train \
    --rl_library rsl_rl --task Isaac-Cartpole \
    physics=isaacsim_physx --viz kit
```

ใช้เมื่อ

ต้องการกล้อง/เซนเซอร์ภาพ, RTX, แก้ฉาก USD ใน GUI

ระวัง

แนะนำ 16 GB VRAM — การ์ด 6–8 GB ใช้ Newton ดีกว่า

CHEAT SHEET

คำสั่งเดิม → คำสั่งใหม่

Isaac Lab 2.x (เดิม)	Isaac Lab 3.0 + Newton
`conda activate env_isaaclab`	`cd ~/IsaacLab`
`isaacsim`	`uv run isaaclab list_envs`
`./isaaclab.sh -p …/rsl_rl/train.py`	`uv run isaaclab train --rl_library rsl_rl`
`./isaaclab.sh -p …/rsl_rl/play.py`	`uv run isaaclab play --rl_library rsl_rl`
`--headless`	(ค่าเริ่มต้น) · ดูภาพ: `--viz newton`
`--resume --load_run RUN`	`--checkpoint latest | best | path`
`--use_pretrained_checkpoint`	`--checkpoint pretrained`
`--video`	`uv run --extra video … --video`

ที่มา: Isaac Lab “Migrating to 3.0” → Reinforcement Learning CLI Entrypoints

TROUBLESHOOTING

เจอบ่อยตอนติดตั้ง

ค้างที่ “Starting physics”

task นั้นตั้งค่าเริ่มต้นเป็น PhysX → เติม physics=newton_mjwarp

เจอจริงตอนทดสอบ (Spot)

Warp CUDA error 999 … gl_buffer

ขั้นตอนเปิด --viz newton บนโน้ตบุ๊ก — viewer ยังทำงานปกติ

เจอจริงตอนทดสอบ · ไม่ต้องแก้

--video แล้วฟ้อง moviepy

รันด้วย uv run --extra video …

CUDA / driver error

อัปเดต NVIDIA driver ≥ 580 แล้วรีบูต · เช็คด้วย nvidia-smi



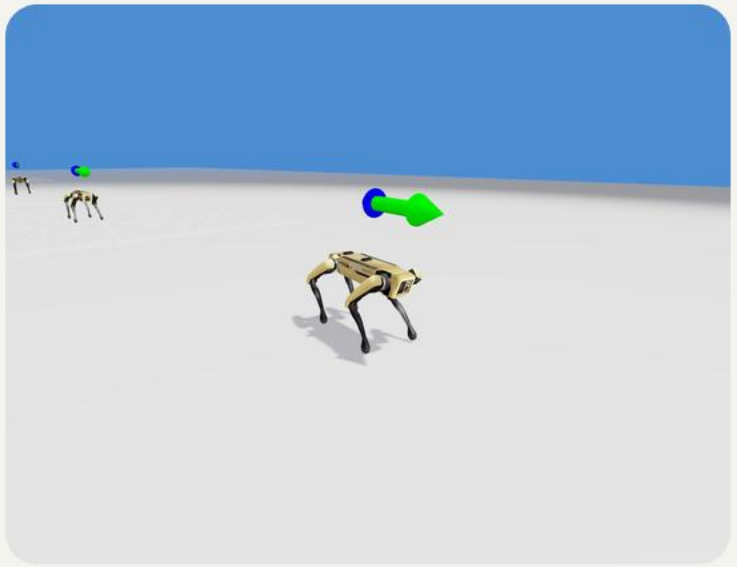
PART 2 · ระหว่างรอดาวน์โหลด

Introduction to Humanoid

Week 1

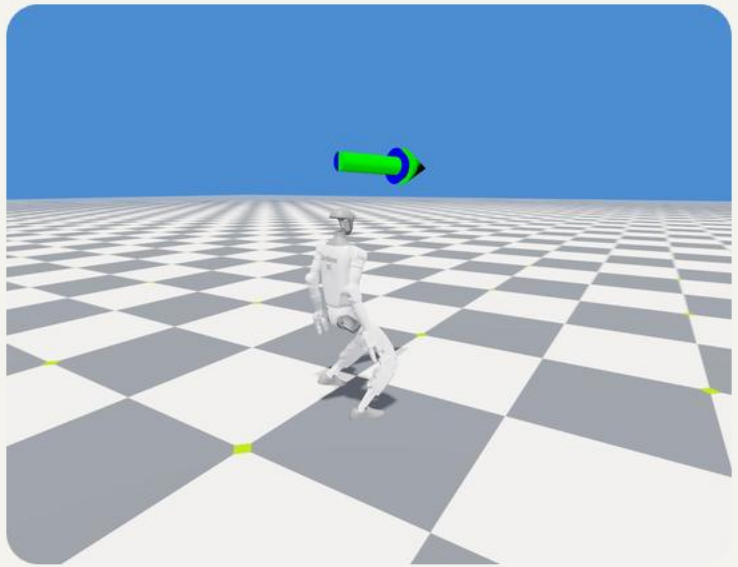
WHAT YOU WILL LEARN

จากหุ่นที่ยืนไม่ได้ → เดินบนพื้นขรุขระ



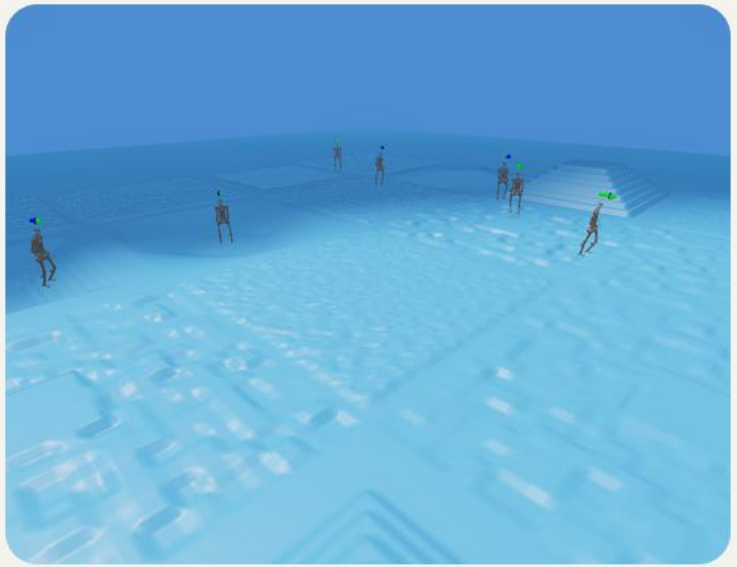
Simulation basics

Isaac Lab 3 + Newton · train / play



Humanoid in sim

G1 / H1 · ขับด้วยคีย์บอร์ด



Reinforcement Learning

reward · terrain · curriculum

ภาพทั้งหมดแคปจากเครื่องทดสอบ · Newton GL viewer

WORKSHOP ROADMAP

จากติดตั้ง → หุ่นที่ทนทาน

Week 1–3	Week 4–7	Week 8–11	Week 12–15
Humanoid in simulator	**RL fundamentals**	**Robustness**	**Toward sim-to-real**
ติดตั้ง · เทรน Spot · G1 + keyboard	ฉาก · reward · terrain · curriculum	ปรับหุ่น · Slopes · Walls · Gaps	domain randomization · ONNX · deploy

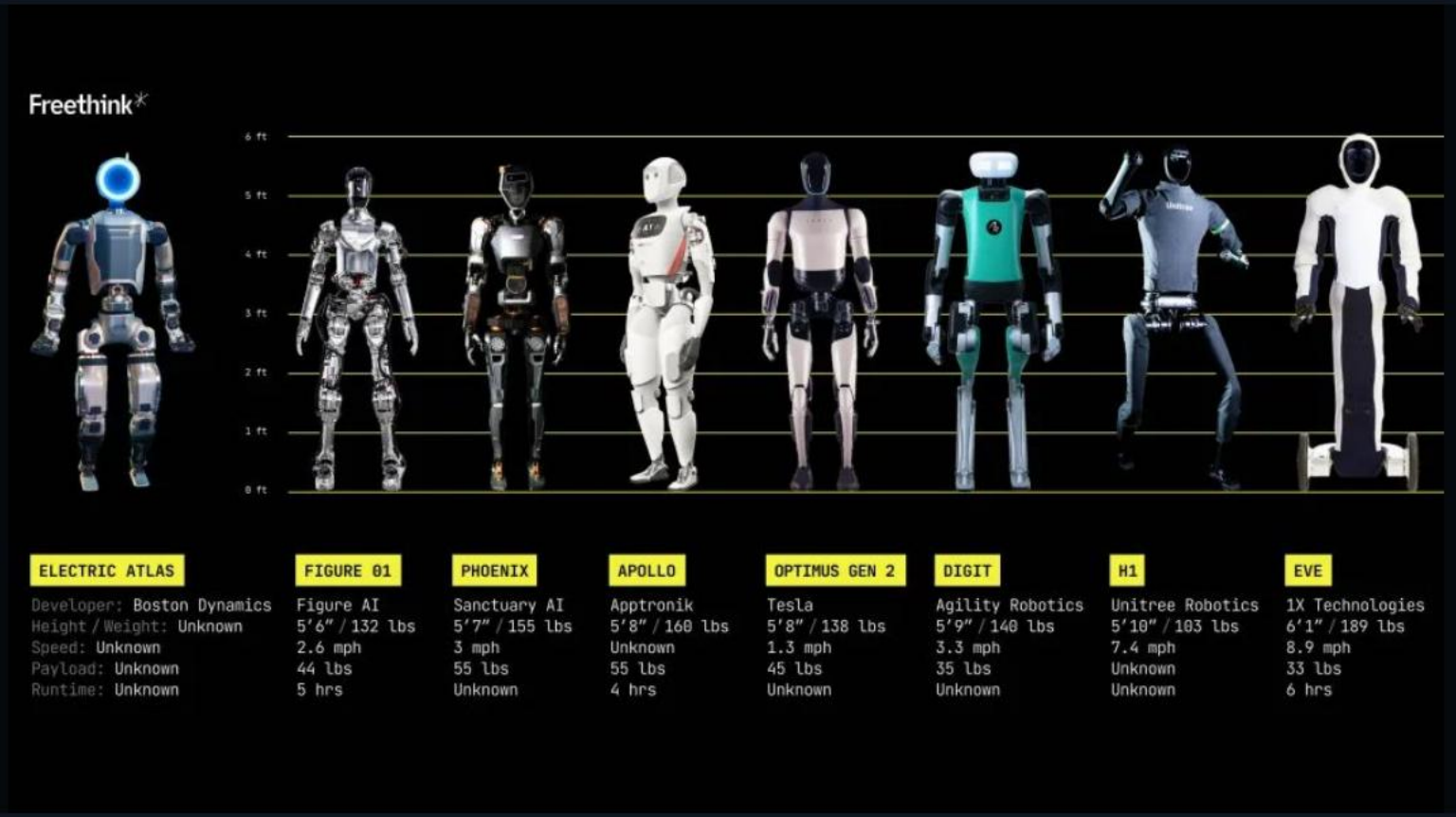
ทุกสัปดาห์จบด้วย **exercises** ที่รันบนเครื่องทดสอบจริงแล้ว — ใช้เช็คว่าเข้าใจ ไม่ใช่แค่ทำตาม

DEFINITION

Human + oid

-oid = มีรูปร่างคล้าย...

humanoid robot = หุ่นยนต์รูปร่างคล้ายคน



ภาพ: freethink.com/robots-ai/humanoids

INTRODUCTION TO ROBOTICS

หุ่นยนต์มีไว้ทำงานแบบ 4D

Dull

งานซ้ำ น่าเบื่อ

Dirty

งานสกปรก

Dangerous

งานอันตราย

Dear

งานต้นทุนแพง

Buzzword ตอนนี้: **Embodied AI / Physical AI**



THE CORE LOOP OF ROBOTICS

Sense → Think → Act

Sense

รับรู้โลก: IMU, encoder ข้อต่อ, กล้อง, height scan

`observations`

Think

ตัดสินใจ — ใน workshop นี้คือ neural-network policy

`policy(obs)`

Act

สั่งมอเตอร์ทุกข้อต่อ แล้ววนกลับไป Sense

`actions → joints`

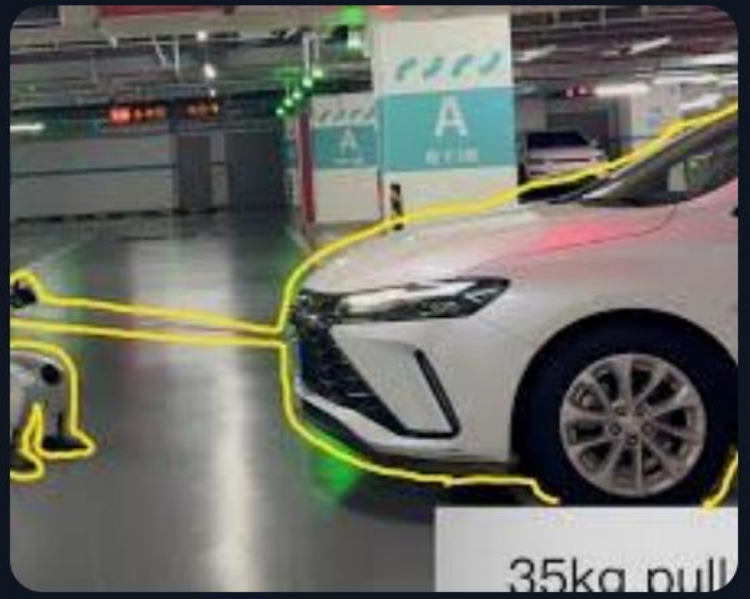
ใน Isaac Lab ลูปนี้วิ่ง 50 ครั้ง/วินาที (step_dt = 0.02 s) พร้อมกันหลายพัน env บน GPU

SHOWREELS

ตอนนี้หุ่นทำอะไรได้แล้วบ้าง?



Agile whole-body motion



Strength: ลากรถได้



Expressive character robots

เบื้องหลังเกือบทั้งหมด: เทรนใน simulation ด้วย RL แล้วย้ายลงหุ่นจริง (sim-to-real)

WHY?

ทำไมหุ่นยนต์ แล้วทำไมต้องรูปร่างคน

Why robotics?

- ขาดแรงงาน
- สังคมสูงวัย
- ประสิทธิภาพ
- สภาพแวดล้อมโหด
- 4D: Dull · Dirty · Dangerous · Dear

Why humanoid?

- โลกออกแบบมาให้คนใช้ — บันได ประตู เครื่องมือ
- หลายทักษะในร่างเดียว
- เรียนจากท่าทางมนุษย์ได้ตรง ๆ

Why simulation?

21,538

steps/วินาที ตอนเทรน Spot 2,048 ตัวพร้อมกันบนโน้ตบุ๊ก

12 นาที

300 iterations · reward 1.6 (iter 19) → 132.7

0 บาท

ค่าซ่อม ไม่ว่าหุ่นจะล้มกี่ครั้ง

ตัวเลขจริง: IsaacContrib-Velocity-Flat-Spot · Newton · RTX 3060 Laptop 6 GB

SIMULATORS IN 2026

มีให้เลือกเยอะ — เลือกตามงาน

Robotics sims

- Gazebo
- Isaac Sim
- Webots
- CoppeliaSim

GPU physics for RL

- **Newton** ← workshop นี้
- MuJoCo · MJX · MuJoCo Warp
- PhysX
- Genesis

Game engines

- Unity
- Unreal Engine

Task-specific

- CARLA (รถ)
- AirSim (โดรน)

Newton = GPU physics engine แบบ open-source บน NVIDIA Warp · เริ่มโดย NVIDIA, Google DeepMind และ Disney Research · โครงการของ Linux Foundation · solver หลักคือ MuJoCo Warp

CHECKPOINT · ก่อนไปต่อ

Lab 1 ผ่านไหม?

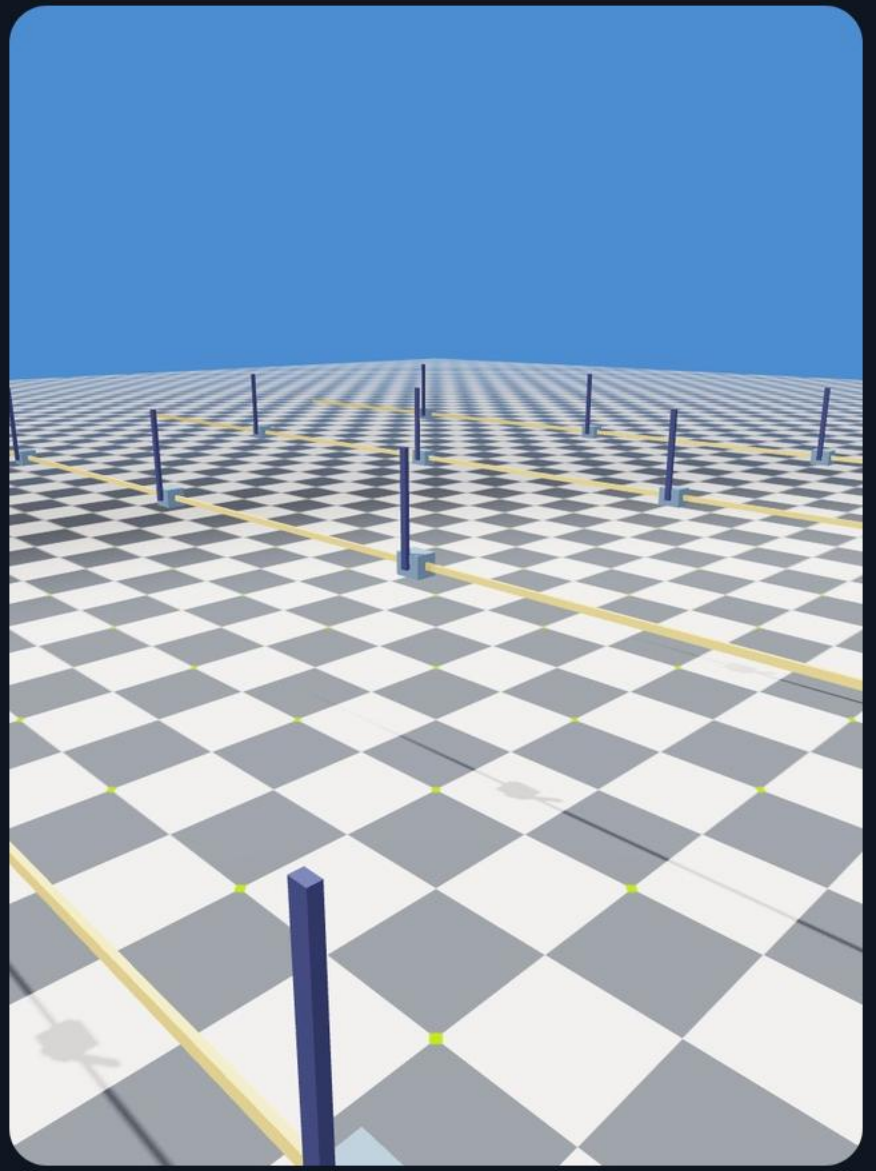
1.1 · Train Cartpole บน Newton

terminal ต้องเห็น newton_mjwarp และ “Training time”

1.2 · Play ด้วย --viz newton

Cartpole ทรงตัวได้ในหน้าต่าง viewer

ผ่าน → ไปทำ Exercises ต่อ · ไม่ผ่าน → ดู Troubleshooting



EXERCISES · WEEK 1

ลองมือกับ Cartpole

4 โจทย์สั้น ๆ เทรนเสร็จในไม่ถึงนาที — ทุกคำสั่งรันบนเครื่องทดสอบแล้ว เทียบผลของคุณกับตัวเลขในสไลด์ได้เลย

EXERCISE 1.1 · GPU SCALING

หุ่นเยอะขึ้น 16 เท่า ช้าลงไหม?

1.1

```
$ uv run isaaclab train --rl_library rsl_rl --task Isaac-Cartpole \
    --num_envs 256 --max_iterations 50 --run_name scale256
# ทำซ้ำด้วย 1024 และ 4096 แล้วจด Steps per second / Training time
# 1 iteration = num_envs × 16 steps → 50 iterations ที่ 4096 envs เก็บข้อมูลมากกว่า 256 envs 16 เท่า
```

num_envs	steps/s	time	ep. len
256	31,075	10.4 s	253
1,024	87,689	10.2 s	300
4,096	383,527	9.8 s	268

โจทย์

1. รันครบ 3 ค่า แล้วจดตัวเลขเทียบกับตาราง
2. เปิด **nvidia-smi -l 1** อีกหน้าต่าง ดู GPU util กับ VRAM
3. **real** ของคำสั่ง **time** ≠ Training time — ส่วนต่างคือเวลา startup

คิดต่อ: เวลาแทบเท่าเดิมเพราะ GPU step ทุก env พร้อมกัน — ลองหาจุดที่เริ่มช้าลง (16384? 65536?) และดูว่า VRAM เต็มก่อนหรือเปล่า

เฉลย · EXERCISE 1.1

steps/s โตตามจำนวน env — จนถึงจุดหนึ่ง

1. ตัวเลขควรใกล้ตาราง: 256 → 31k, 1,024 → 88k, 4,096 → 384k steps/s ที่เวลาเทรน ~10 s เท่ากัน
2. บนเครื่องทดสอบ nvidia-smi ใช้ไม่ได้ (driver อัปเดตแต่ยังไม่รีบูต) — ถ้าเครื่องคุณใช้ได้ จะเห็น GPU util สูงขึ้นตาม num_envs
3. real ≈ 25 s แต่ Training time ≈ 10 s → ~15 s คือ startup (โหลด Python, สร้างฉาก, compile kernel) · รอบแรกนานกว่า (43 s) เพราะ cache ยังว่าง

num_envs	steps/s	time
4,096	384k	9.8 s
16,384	1.26 M	11.9 s
65,536	2.26 M	23.9 s

50 iterations · Cartpole · RTX 3060 6 GB — 65,536 ยังรันได้ แต่เวลาเริ่มเพิ่ม = GPU เต็มแล้ว

สรุป: ยิ่ง env เยอะ ยิ่งได้ sample ต่อวินาทีมาก แต่พอ GPU อิ่ม เวลา/iteration จะเริ่มโต — เลือก num_envs ที่ยังอยู่ช่วงเวลาคงที่

EXERCISE 1.2 · HYDRA OVERRIDE

ลดตัวคูณ action → แรง 10 เท่า จะเป็นไง?

1.2

```
$ uv run isaaclab train --rl_library rsl_rl --task Isaac-Cartpole \
    --run_name effort10 env.actions.joint_effort.scale=10.0
# scale = ตัวคูณ action → แรง (N) ของข้อ slider_to_cart: action 1.0 = 100 N (เดิม) → 10 N
# เพดานแรงของมอเตอร์ (joint_effort_limit = 400 N) ไม่เปลี่ยน · รูปแบบ env.<path> = ค่า ตาม config class · agent.<path> สำหรับ PPO
```

ผลบนเครื่องทดสอบ · RTX 3060 LAPTOP 6 GB

4.93 → 3.09

Mean reward หลัง 150 iterations (scale 100 → 10)

300 → 231

Mean episode length (เต็ม = 300 · ต่ำ = ล้มก่อนหมดเวลา)

โจทย์

1. ดูชื่อ field จริงใน **cartpole_manager_env_cfg.py → ActionsCfg.joint_effort.scale**
2. ลอง 30, 300 — ค่าไหนเริ่ม “พอ”?
3. ลอง **agent.seed=1** ซ้ำ 2 รอบ แล้วดูว่าผลต่างกันแค่ไหน

คิดต่อ: เพดานแรงยัง 400 N แต่ต้องสั่ง action ใหญ่ขึ้น 10 เท่า จึงเรียนยากขึ้น · มอเตอร์อ่อนจริง = ลด joint_effort_limit

เฉลย · EXERCISE 1.2

ตัวคูณ action ต้องเหมาะก่อน RL ถึงเรียนได้ดี

1. field อยู่ที่ **ActionsCfg.joint_effort** = **JointEffortActionCfg(..., scale=100.0)** → Hydra path คือ **env.actions.joint_effort.scale**
2. ดูตาราง: 30 เกือบพอ, 100 (ค่าเดิม) กับ 300 ทรงตัวได้เต็ม 300 step — มากกว่านี้ไม่ได้ช่วยแล้ว
3. seed เดียวกัน 2 รอบได้ **ตัวเลขตรงกันทุก iteration** (deterministic บนเครื่องเดียว) · seed ต่างกันได้ curve ต่าง เช่น iteration 19: 1.62 (seed 1) vs 0.71 (seed 42 ค่าเริ่มต้น)

scale	reward	ep. len
10	3.09	231
30	4.60	280
100 (เดิม)	4.93	300
300	4.95	300

150 iterations · 4,096 envs

สรุป: เวลาเทียบ config ให้ล็อก seed — ไม่งั้นแยกไม่ออกว่าผลต่างมาจาก config หรือจากโชค

EXERCISE 1.3 · OBSERVATION ABLATION

ถ้า policy มองไม่เห็นความเร็ว?

1.3

```
$ uv run isaaclab train --rl_library rsl_rl --task Isaac-Cartpole \
    --run_name novel env.observations.policy.joint_vel_rel=null
# obs 4 ค่า = ตำแหน่ง (cart, เสา) + ความเร็ว (cart, เสา) · rel = เทียบกับท่าเริ่มต้น
# null = ลบ term ทั้งก้อน → ความเร็วหายทั้ง cart และเสา (4 → 2 ค่า)
```

ผลบนเครื่องทดสอบ · RTX 3060 LAPTOP 6 GB

4.93 → 2.83

Mean reward · 150 iterations

300 → 206

Mean episode length

โจทย์

1. ปิด **joint_pos_rel** แทน แล้วเทียบ — อันไหนสำคัญกว่า?
2. ดูตาราง **Observation Manager** ตอนเริ่มเทรน: shape (4,) → (2,)
3. **null** = ลบ term ออกจาก config (Hydra)

คิดต่อ: ตำแหน่งอย่างเดียวบอกไม่ได้ว่าเสากำลังล้มไปทางไหน — state ไม่ครบ (not Markov) หุ่นจริงจึงต้องมี IMU/encoder ความเร็ว หรือใช้ history ของ observation

เฉลย · EXERCISE 1.3

ทั้งตำแหน่งและความเร็วจำเป็น — ขาดอย่างไหนก็ไม่ครบ

1. ปิด **joint_pos_rel**: reward 2.27 · ep. len 233 — แย่พอ ๆ กับปิดความเร็ว (2.83 · 206) เพราะ policy ไม่รู้ว่าเสาเอียงอยู่ตรงไหน
2. ตาราง **Observation Manager** ตอนเริ่มเทรน: shape (4,) → (2,) เหลือแค่ term เดียว — ใช้เช็คทุกครั้งว่า override ทำงานจริง
3. **null** ใน Hydra = None → term ถูกลบออกจาก config ก่อนสร้าง env (ไม่ใช่ใส่ค่า 0)

obs	dim	reward	ep. len
ครบ (เดิม)	4	4.93	300
ไม่มีความเร็ว	2	2.83	206
ไม่มีตำแหน่ง	2	2.27	233

สรุป: observation ต้องบอก state ให้ครบ — หุ่นจริงมี encoder (ตำแหน่ง) + IMU/ความเร็วข้อต่อ ด้วยเหตุผลเดียวกัน

EXERCISE 1.4 · ANOTHER RL LIBRARY

task เดิม เปลี่ยนแค่ library

1.4

```
$ uv run --extra skrl isaaclab train --rl_library skrl --task Isaac-Cartpole
# log อยู่ที่ logs/skrl/cartpole/<run>/ (TensorBoard)
# --extra skrl = ติดตั้ง skrl เพิ่ม · 2,400 timesteps = 150 update × 16 rollouts = เท่ากับ 150
iterations ของ rsl_rl
```

library	ความยาว	เวลา	reward	ep. len
rsl_rl	150 iterations	31.6 s	4.93	300
skrl	2,400 timesteps	≈ 39 s	4.95	300

โจทย์

1. rsl_rl พิมพ์ผลลง terminal · skrl เขียนลง TensorBoard
2. เปิดดู: **uv run tensorboard --logdir logs/skrl/cartpole**
3. เทียบกราฟ reward ของสอง library บนแกนเวลาเดียวกัน

คิดต่อ: environment เดียวกันใช้ได้กับ rsl_rl · skrl · sb3 · rl_games — เวลาเทียบ algorithm ให้ล็อก env, seed และจำนวน sample ให้เท่ากัน

เฉลย · EXERCISE 1.4

env เดียวกัน library ต่างกัน ผลใกล้กันมาก

1. rsl_rl พิมพ์ตาราง Mean reward ลง terminal **และ** เขียน TensorBoard · skrl แสดงแค่ progress bar (2,400 timesteps) แล้วเขียนผลลง TensorBoard อย่างเดียว
2. **uv run tensorboard --logdir logs** เห็นทั้ง logs/rsl_rl และ logs/skrl ในหน้าเดียว · ชื่อกราฟต่างกัน: rsl_rl = Train/mean_reward, skrl = Reward / Total reward (mean)
3. ผลสุดท้ายแทบเท่ากัน (4.93 vs 4.95, ep. len 300 ทั้งคู่) แต่ skrl ใช้เวลามากกว่า ~25% (39 s vs 31.6 s) ในการตั้งค่า default ของแต่ละตัว

สรุป: เลือก library จาก ecosystem ที่ทีมใช้ — แต่เทียบให้ยุติธรรม: env, seed, จำนวน sample และเวลาเท่ากัน